
Hardware-Aware Optimization and Benchmarking of Generative Transformers on Edge ARM Architectures

Daniel Sanei

Department of Electrical and Computer Engineering
University of California, San Diego
dsanei@ucsd.edu

Abstract

This paper presents a hardware-aware benchmarking study of autoregressive language model inference on the Raspberry Pi 5, a commodity single-board computer.¹ We evaluate four transformer architectures spanning 1B to 8B parameters across two post-training quantization levels, three context window sizes, and two thread counts, simultaneously profiling decoding throughput, generative fidelity, and hardware telemetry including Power Management Integrated Circuit (PMIC) power draw, die temperature, clock frequency, and memory pressure. Our central finding is that autoregressive decoding on the Pi 5’s Cortex-A76 is fundamentally memory-bandwidth-bound: the LPDDR4X bus, not the processor’s computational capacity, is the binding constraint across all 202 configurations tested. Doubling the thread count reduces throughput, and the quality cost of 4-bit quantization varies from negligible to catastrophic depending on architecture. These results provide a concrete characterization of the performance and reliability envelope for local LLM deployment on modern low-power ARM edge hardware.

1 Research Problem

This work investigates the computational challenges of deploying autoregressive transformer models on edge hardware, where these models must compute complex joint probability distributions $P(x_1, \dots, x_T)$ under strict memory and power constraints. The Raspberry Pi 5 integrates a VideoCore VII GPU within its BCM2712 SoC, but no mature inference framework currently targets it for transformer workloads; all computation therefore falls to the four Cortex-A76 CPU cores. This project evaluates the performance floor of these generative distributions by applying post-training quantization (PTQ) and measuring how reduced-precision weight representations affect both generative fidelity and hardware-level stability across a systematic sweep of models and configurations.

2 Importance

As generative AI moves toward local deployment in robotics and embedded systems, understanding how to run these models on low-power silicon is critical. Since typical desktop GPUs are unsuitable for portable applications, we must optimize DGM architectures to run locally. This work helps define hardware-aware requirements for edge-deployed generative applications by analyzing the efficiency of lightweight ARM processors in sustaining transformer inference workloads, without cloud infrastructure and the associated latency and privacy trade-offs.

¹Code and data are publicly available at https://github.com/danielsanei/edge_ai_performance_profiling (5).

3 Existing Works and Limitations

Existing research typically focuses on deploying transformers on high-end desktop GPUs or Apple Silicon. While works deploying lightweight models on single-board computers (SBCs) via quantization exist, they prioritize raw accuracy over hardware-level efficiency on ARM architectures. Nezami et al. (1) identified bottlenecks in the decode phase on Raspberry Pi 5 clusters that limit scalability for large context windows. Yoon et al. (2) demonstrated INT8 quantization on the Pi 4 but lacked ARM-specific cache analysis and power profiling. Unlike prior work, our study simultaneously captures ARM clock frequency, PMIC power rails, die temperature, and CPU throttle status alongside inference metrics, providing a unified multi-signal hardware profile across a systematic sweep of quantization, context window, and thread-count configurations.

4 Inference Framework and Software Stack

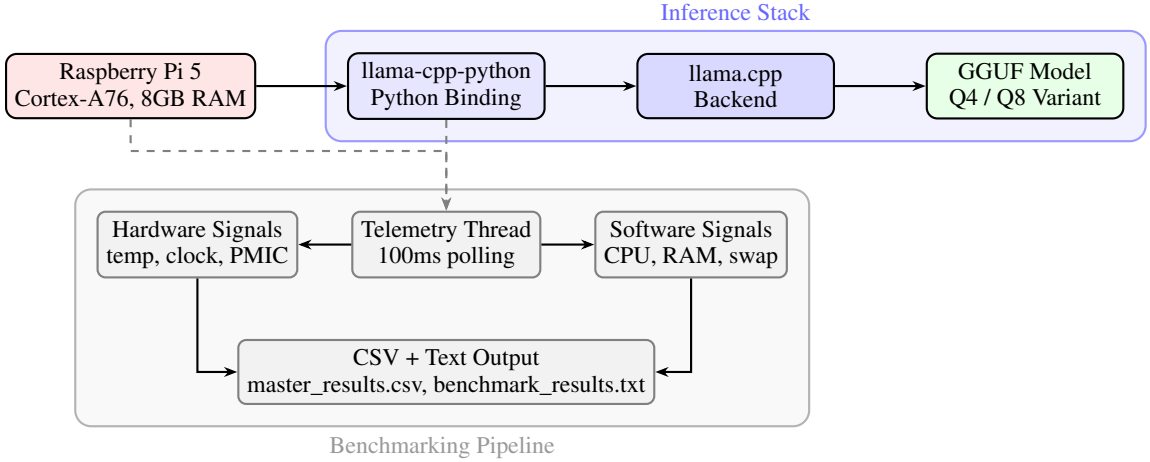


Figure 1: System architecture. The inference stack (llama-cpp-python → llama.cpp → GGUF) runs on the Pi 5 while a parallel telemetry thread samples hardware and software signals every 100 ms.

4.1 The llama-cpp-python Backend

We use llama-cpp-python (4), a Python binding for llama.cpp, an open-source C/C++ inference engine optimized for consumer CPU inference via BLAS and SIMD intrinsics, with native support for quantized model formats. Unlike higher-level wrappers such as Ollama, it exposes per-token logit scores directly, enabling unified computation of throughput and perplexity in a single pipeline.

Model weights are stored in GGUF (GPT-Generated Unified Format), a single-file container for quantized weights, metadata, and tokenizer data suited to unified memory architectures like the Pi 5. Weights are mmap-loaded, so reported RAM captures only KV cache and compute buffers, not the weight file itself. The 8 GB ceiling bounds practical model size: a 7B Q8 model occupies 7.2 GB, leaving under 1 GB for runtime overhead, while Q4 at 4.1 GB provides comfortable headroom (KV cache discussed in Section 5.2).

4.2 Model Selection

Model selection was primarily constrained by the Pi 5’s 8 GB RAM ceiling, which bounds the practical deployment range to models under approximately 8 B parameters at Q4 quantization (or under 4 B at Q8), ruling out larger architectures. We selected four models spanning the feasible parameter range, summarized in Table 1. Llama 3.2 (1B) serves as a throughput baseline; Phi-4 mini (3.8B) tests a reasoning-optimized architecture; Mistral 7B v0.3 introduces grouped-query and sliding window attention, with its Q8 footprint (7.2 GB) approaching the platform ceiling; and Llama 3.1 (8B) stress-tests the upper limit, with the Q8 variant excluded entirely as its 8.0 GB weight file saturates the full 8 GB before any KV cache or OS overhead.

Model	Params	Q4 Size	Q8 Size	Role
Llama 3.2	1B	0.7 GB	1.3 GB	Throughput baseline
Phi-4 mini	3.8B	2.4 GB	3.9 GB	Reasoning-optimized
Mistral 7B v0.3	7B	4.1 GB	7.2 GB	Architecture diversity
Llama 3.1	8B	4.6 GB	8.0 GB*	Upper boundary

Table 1: Model selection and approximate GGUF memory footprints. *Llama 3.1 8B Q8 was excluded: its weight file alone saturates the 8 GB ceiling before any runtime overhead.

5 Methods

5.1 Mathematical Setup

The models under evaluation are autoregressive decoder-only transformers that factorize the joint probability distribution of a sequence as:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}; \theta) \quad (1)$$

Token generation involves iterative sampling from the Softmax output, controlled by temperature τ :

$$P(x_t = i \mid x_{<t}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (2)$$

Our benchmarks evaluate the hardware’s ability to compute these distributions across billions of parameters in real-time, focusing on sustained throughput during autoregressive decoding.

5.2 Quantization and KV Cache

Following Dettmers et al. (3), we apply Post-Training Quantization (PTQ) to reduce 16-bit floating-point weights to lower-precision integers. We compare two tiers: Q8_0 (uniform 8-bit) and Q4_K_M (4-bit mixed-precision block quantization), using the standard linear mapping:

$$w_q = \text{round} \left(\frac{1}{s} \cdot w_f + z \right) \quad (3)$$

To optimize decoding, `llama.cpp` employs a Key-Value (KV) Cache that stores self-attention keys and values across generation steps, reducing per-step attention from $O(n^2)$ to $O(n)$ per new token. The cache grows linearly with sequence length, introducing memory pressure that compounds with model size. We monitor anonymous RAM growth to evaluate whether the 8 GB ceiling can sustain longer generations without swap.

5.3 Inference Configuration Parameters

In addition to quantization, `llama.cpp` exposes runtime parameters set at model initialization:

```
llm = Llama(model_path="model.gguf",
            n_ctx=2048, n_threads=4,
            type_k=8, logits_all=True,
            verbose=False, seed=42)
```

Context Window (`n_ctx`) controls KV cache allocation; we evaluate 512, 2048, and 4096 tokens. **Thread Count** (`n_threads`) controls how many Cortex-A76 cores are used; we compare 2 and 4 threads. Since decoding is memory-bandwidth-bound, additional threads are expected to increase bus contention rather than throughput. **KV Cache Quantization** (`type_k=8`) reduces cached key tensors to INT8, lowering KV cache memory pressure relative to FP16 — critical when running 7–8B models near the 8 GB ceiling.

5.4 Generative Fidelity and Perplexity

To quantify the quality cost of compression, we use Perplexity (PPL), the exponentiated average negative log-likelihood:

$$\text{PPL}(X) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i | x_{<i}) \right) \quad (4)$$

PPL is computed using `llama-cpp-python`’s native evaluation interface, which provides direct access to the model’s internal logit tensor without going through a high-level API. The input text is tokenized via `llm.tokenize()`, then a forward pass is executed via `llm.eval()`. Raw logits are retrieved from the model’s internal score buffer (`llm._scores`), and a numerically stable softmax is applied per time-step to extract the log-probability of each ground-truth token; the scalar PPL is then computed from the mean log-likelihood. This ensures numerical consistency across all model and quantization combinations without modifying the underlying weights.

5.5 Performance Metrics and Telemetry

Per run, we record Time to First Token (TTFT) (prefill latency), Tokens Per Second (TPS) (sustained decoding throughput), and peak RAM. A parallel Python thread samples hardware state every 100 ms using `vcgencmd`, a Raspberry Pi-specific command-line utility that communicates directly with the VideoCore firmware, exposing hardware telemetry not accessible through standard Linux interfaces. Timing metrics use `time.perf_counter()` wall-clock timestamps for sub-millisecond resolution.

- **Power and Thermals:** `vcgencmd pmic_read_adc` reads current and voltage from all rails of the board’s PMIC—the dedicated chip that regulates power delivery to the CPU, RAM, and USB, allowing total board wattage to be estimated as $P = \sum I_i V_i$. Die temperature is read via `vcgencmd measure_temp`, and throttle status via `vcgencmd get_throttled`, which returns a bitmask encoding whether the processor has entered soft throttling ($\geq 80^\circ\text{C}$) or hard throttling ($\geq 85^\circ\text{C}$).
- **Clock and CPU:** `vcgencmd measure_clock arm` returns the current ARM core frequency, enabling detection of governor-driven downscaling events. System CPU utilization and anonymous memory consumption are sampled via `psutil`.

6 Experimental Design

All tests use a fixed random seed and `temperature=0.0` for deterministic, reproducible outputs. Table 2 summarizes the experimental sweep.

Axis	Values
Models	Llama 3.2 (1B), Phi-4 mini (3.8B), Mistral 7B v0.3, Llama 3.1 (8B)
Quantization	Q4_K_M, Q8_0
Context window	512, 2048, 4096 tokens
Thread count	2, 4
Prompts	P1–P5

Table 2: Experimental sweep. Of 240 theoretical configurations, 35 were excluded before execution: 30 for Llama 3.1 8B Q8_0 (weight file alone saturates RAM) and 5 for Mistral 7B Q8_0 at `n_ctx=4096`, `n_threads=4` (excluded after the 2-thread variant triggered disk swap).

The five prompts used across all configurations are:

1. **P1 (Technical):** “Is Python an interpreted or compiled language?”
2. **P2 (Creative Writing):** “Write a short story about a robot discovering a garden on a deserted planet.”
3. **P3 (Summarization):** “Summarize the key differences between ARM and x86 architectures in two paragraphs.”

4. **P4 (Logical Reasoning):** “A farmer has 17 sheep. All but 9 run away. How many sheep does the farmer have left? Explain your reasoning step by step.”
5. **P5 (Mathematical Reasoning):** “Derive the formula for the sum of an arithmetic series and verify it for the integers 1 through 10.”

This range tests whether inference characteristics differ across generative modes and provides a diverse basis for perplexity comparison.

7 Results and Analysis

Of the 205 planned configurations (after pre-execution exclusions), 202 generated results. Three configurations—Mistral 7B Q8_0 at $n_ctx=4096$, $n_threads=2$, prompts P3–P5—were never run after P1 and P2 exhausted physical RAM. Results are organized by inference dimension: throughput hierarchy, memory-bandwidth sensitivity, context window effects, generative fidelity, and thermal/power behavior.

7.1 Throughput Hierarchy

Table 3 reports baseline decoding throughput for all model–quantization combinations at the conservative configuration ($n_ctx=512$, $n_threads=2$). Values use prompt P2 to exclude the first-prompt page-fault warm-up effect present in large Q8 models, discussed in Section 7.5.

Model	Quant	TPS	Q4/Q8 Speedup
Llama 3.2 (1B)	Q4_K_M	12.99	1.48×
	Q8_0	8.78	
Phi-4 mini (3.8B)	Q4_K_M	4.10	1.40×
	Q8_0	2.92	
Mistral 7B v0.3	Q4_K_M	2.59	1.68×
	Q8_0	1.54	
Llama 3.1 (8B)	Q4_K_M	2.19	—

Table 3: Baseline decoding throughput ($n_ctx=512$, $n_threads=2$, prompt P2). TPS shows minimal variation across prompts P2–P5 for a fixed configuration, confirming the decode phase is semantically agnostic and purely bandwidth-driven.

TPS scales inversely with model weight size, and the Q4/Q8 speedup grows with model size, reflecting proportionally greater memory pressure of larger Q8 footprints. Prefill throughput follows the same hierarchy, with TTFT ranging from under 2 s for Llama 1B to nearly 6 s for Llama 3.1 8B at the longest prompt. Figure 2 visualizes the quantization trade-off across both throughput and fidelity dimensions simultaneously.

7.2 The Memory-Bandwidth Bound: Thread Count Ablation

A core hypothesis is that autoregressive decoding is memory-bandwidth-bound on the Cortex-A76: each decode step streams the full weight tensor through the LPDDR4X bus regardless of how many cores are active, so additional threads increase bus contention without increasing available bandwidth. If true, throughput should *decrease* when thread count is doubled. Table 4 confirms this across every model–quantization combination.

Figure 3 shows the degradation consistently across all model–quantization pairs. Larger models suffer steeper penalties due to heavier per-core bus demands, and Q8 configurations degrade more than their Q4 counterparts for the same model. The practical implication is clear: 2 threads strictly dominates 4 in throughput, power draw, and thermal load across all configurations tested.

7.3 Context Window and KV Cache Pressure

Table 5 shows the effect of increasing context window on TPS across all Q4 models at 2 threads. The impact is modest: TPS decreases by only 1–11% from $n_ctx=512$ to 4096.

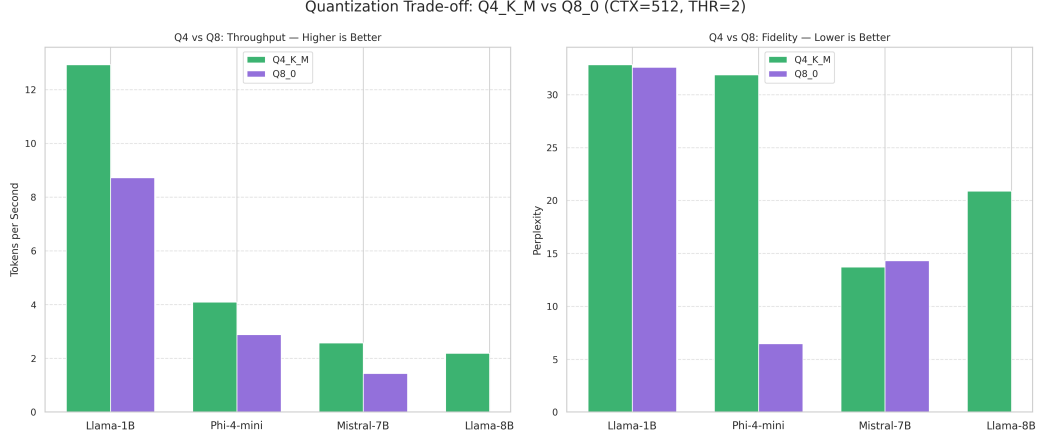


Figure 2: Quantization trade-off at baseline ($n_ctx=512$, $n_threads=2$). Left: Q4_K_M consistently delivers higher TPS. Right: The PPL cost of Q4 is model-dependent—negligible for Llama 1B, severe for Phi-4 mini, and counterintuitively lower for Mistral 7B (see Section 7.4).

Model	Quant	2-Thr TPS	4-Thr TPS	Δ
Llama 3.2	Q4	12.99	9.94	−23.5%
	Q8	8.78	6.24	−28.9%
Phi-4 mini	Q4	4.10	3.32	−19.0%
	Q8	2.92	2.07	−29.1%
Mistral 7B	Q4	2.59	2.03	−21.6%
	Q8	1.54	1.21	−21.4%
Llama 3.1	Q4	2.19	1.50	−31.5%

Table 4: Effect of doubling thread count on decoding TPS ($n_ctx=512$, prompt P2). Every configuration shows a 19–32% throughput reduction, empirically confirming that the LPDDR4X memory bus is the binding constraint.

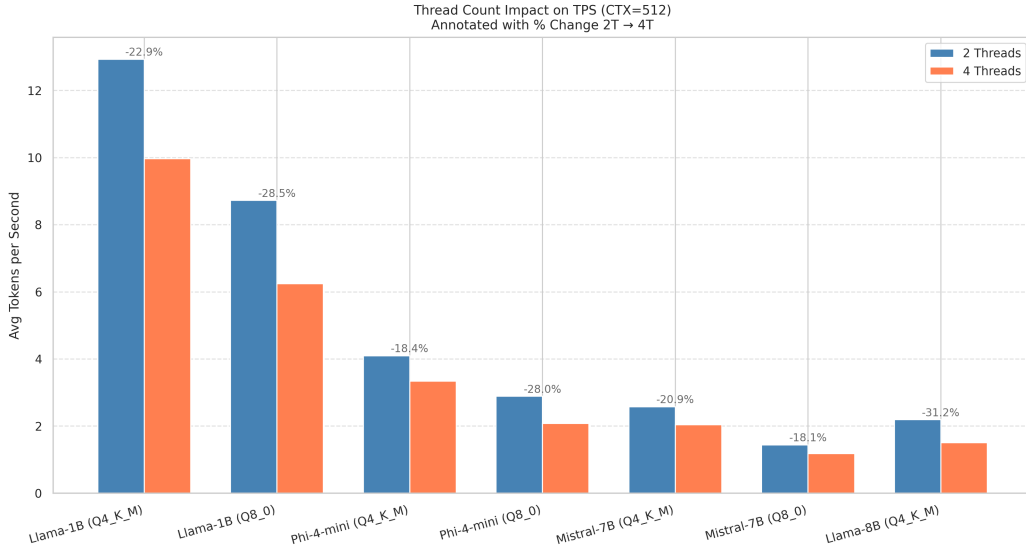


Figure 3: 2-thread vs. 4-thread TPS at $n_ctx=512$. Percentage annotations show consistent throughput degradation when doubling thread count across all configurations.

The small TPS effect arises because all prompts generate at most 128 tokens, so the active KV cache never fills beyond 128 entries during decoding regardless of the allocated window. The memory

Model	ctx=512	ctx=2048	ctx=4096	Δ
Llama 3.2	12.99	12.54	12.58	−3.2%
Phi-4 mini	4.10	4.07	3.74	−8.8%
Mistral 7B	2.59	2.53	2.31	−10.8%
Llama 3.1	2.19	2.19	2.16	−1.4%

Table 5: Decoding TPS vs. context window size (Q4_K_M, n_threads=2, prompt P2). Mistral 7B shows the largest sensitivity, likely due to its sliding window attention architecture.

impact is nonetheless measurable: because weights are mmap-loaded and served from the OS page cache, the reported peak_ram captures only KV cache and compute buffer allocations. Figure 4 provides a visual overview of TPS across all model–context combinations.

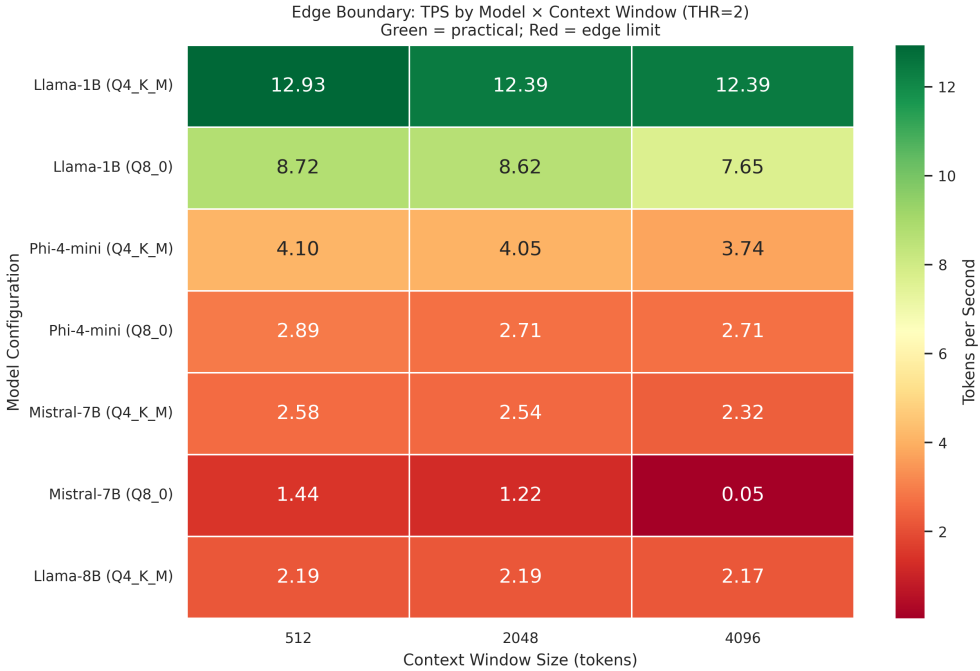


Figure 4: TPS heatmap: model–quantization vs. context window (n_threads=2). The Mistral 7B Q8_0 cell at n_ctx=4096 reflects the disk swap event (0.05 TPS).

7.4 Generative Fidelity Under Quantization

Table 6 reports perplexity for each model–quantization pair across all five prompts. PPL is invariant to n_ctx and n_threads; it is a function of the model’s probability distribution over the prompt sequence alone. Lower is better.

Three distinct patterns emerge. Phi-4 mini is highly sensitive to quantization: Q4 increases PPL by $4.8\times$ on P1 and $10\times$ on P3 while delivering only a $1.40\times$ throughput gain, strongly arguing against Q4 deployment for quality-critical tasks. Mistral 7B shows minimal sensitivity: Q4 and Q8 PPL values differ by less than 10% across all prompts, with Q4 matching or marginally outperforming Q8 on most—a property of Q4_K_M’s mixed-precision block scheme, which preserves high-salience weight blocks at higher effective precision. This makes Mistral 7B Q4 the most favorable configuration in the study: higher throughput at no meaningful quality cost. Llama 3.2 1B shows similarly negligible quantization sensitivity. Across all models, mathematical and logical prompts (P4, P5) yield consistently lower perplexity than technical or creative prompts (P1, P2), reflecting the more constrained token distributions of structured outputs.

Model	Quant	P1 (Tech.)	P2 (Creative)	P3 (Summ.)	P4 (Logic)	P5 (Math)
Llama 3.2 (1B)	Q4_K_M	76.64	29.26	23.31	22.63	12.33
	Q8_0	73.54	30.63	25.01	22.20	11.57
Phi-4 mini (3.8B)	Q4_K_M	58.46	15.23	47.56	10.77	27.38
	Q8_0	12.28	4.91	4.76	4.62	5.83
Mistral 7B v0.3	Q4_K_M	24.97	14.08	13.35	9.62	6.55
	Q8_0	27.71	14.38	13.63	9.20	6.66
Llama 3.1 (8B)	Q4_K_M	43.94	21.66	17.28	12.38	9.24

Table 6: Perplexity across all model–quantization combinations and prompt types. Lower is better. Llama 3.1 8B was evaluated at Q4 only.

7.5 Thermal Behavior, Power Draw, and System Boundary

Thermal tiers. Die temperature forms four clear tiers by model weight size (Figure 5): Llama 3.2 1B peaks at 63–73 °C; Phi-4 mini at 66–83 °C; Mistral 7B at 71–82 °C under normal conditions; and Llama 3.1 8B at 75–85.1 °C, reaching the Pi 5’s rated maximum. Thermal throttling (flagged by `vcgencmd get_throttled`) was triggered in most configurations involving the three largest models.

Power draw. Board-level power spans 6.5–7.3 W across all models—a narrow range reflecting the bus-saturating nature of inference regardless of model size. Because power is approximately constant, energy efficiency (joules per token) is directly proportional to TPS: Llama 3.2 1B Q4 is approximately 6× more energy-efficient than Llama 3.1 8B Q4 on a per-token basis.

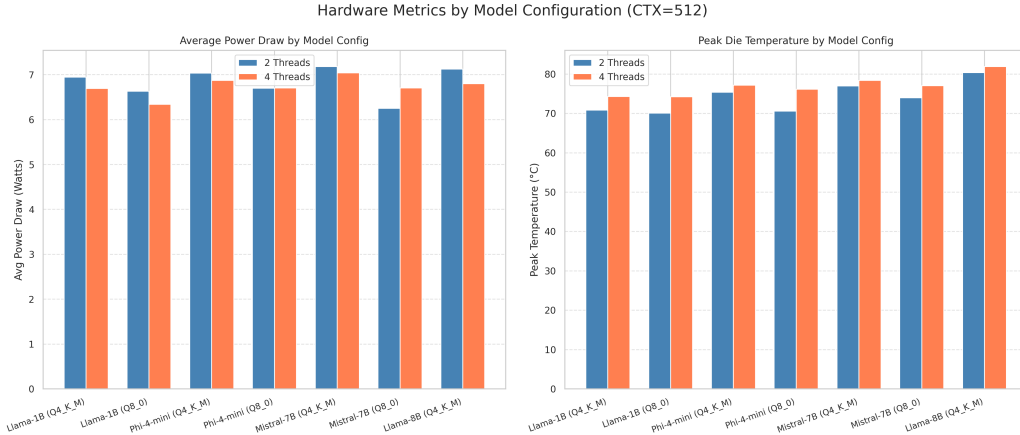


Figure 5: Average power draw (left) and peak die temperature (right) by model–quantization at `n_ctx=512`. Power is narrow (6.5–7.3 W) while temperature follows a clear size-driven tier, with Llama 3.1 8B approaching the Pi 5’s 85 °C maximum.

Mistral Q8 at `ctx=4096`: disk swap event. The Mistral 7B Q8 weight footprint (7.2 GB) combined with KV cache and OS overhead exceeds the 8 GB ceiling at `n_ctx=4096`, forcing the kernel to swap pages to the SD card. Table 7 presents the full multi-signal telemetry for this event.

The diagnostic is unambiguous: 9% CPU utilization indicates a processor blocked on SD card I/O, causing the frequency governor to scale down to 1733 MHz with power and temperature falling accordingly. Each prompt required 39–44 minutes for 121–130 generated tokens. A milder precursor appears at `n_ctx=2048`, where TPS drops to 1.09–1.35 (vs. 1.54 at `ctx=512`) and average CPU falls below 60%, indicating episodic page eviction before the ceiling is fully breached.

A first-prompt cold-start anomaly affects Mistral Q8 across *all* configurations: at P1 of each session, TTFT rises to 58–94 s and TPS falls to 0.69–1.34, as `mmap` triggers a sustained demand-paging wave across the full 7.2 GB weight file. Prompts P2–P5 run at stable throughput once the working set is page-cached.

Signal	Normal (ctx=512)	ctx=4096 P1	ctx=4096 P2
TPS	1.54	0.05	0.05
Total time (s)	~88	2361	2613
Avg power (W)	6.65	2.99	2.95
Avg CPU (%)	58.5	9.2	9.0
Avg clock (MHz)	2398	1736	1730
Peak temp (°C)	~74	57.6	56.0

Table 7: Mistral 7B Q8 telemetry: normal (ctx=512) vs. the ctx=4096 swap event. All six signals deviate simultaneously in a direction consistent with I/O-bound operation. The benchmark was terminated after P2; P3–P5 were never run, and the `n_threads=4` variant was excluded entirely.

8 Conclusion

8.1 Summary of Findings

This study systematically characterizes the performance envelope for deploying autoregressive language models on the Raspberry Pi 5. Across 202 configurations spanning four models, two quantization tiers, three context window sizes, and two thread counts, three principal findings emerge.

The memory wall is empirically confirmed. Every model–quantization combination shows throughput degradation when doubling thread count, with no exceptions. The LPDDR4X bus, not the Cortex-A76 cores, is the binding constraint for autoregressive decoding. This result is directly actionable: 2 threads strictly dominates 4 for all configurations tested.

Quantization is necessary but its quality cost is highly model-dependent. Q4 substantially improves throughput but the fidelity impact ranges from negligible for small models to catastrophic for Phi-4 mini. Mistral 7B v0.3 is the most favorable configuration: Q4_K_M matches Q8_0 in perplexity while delivering significantly higher throughput. Mistral 7B Q4 also outperforms the larger Llama 3.1 8B on both speed and fidelity, demonstrating that architecture—not parameter count alone—determines edge inference performance.

The 8 GB memory ceiling is a hard constraint with a well-defined boundary. Mistral 7B Q8 at ctx=4096 triggers a 30× throughput collapse to 0.05 TPS as the system falls back to SD card swap. The simultaneous deviation of six hardware signals provides a diagnostic fingerprint for this failure mode, illustrating the value of co-profiling hardware and inference metrics beyond simple throughput measurement.

8.2 Limitations

- **CPU-only inference.** All results represent the CPU performance floor; hardware accelerators such as the Hailo-8 AI Hat or Coral Edge TPU were not evaluated.
- **Single-device study.** All benchmarks were conducted on one Pi 5 unit; device-to-device variation in LPDDR4X binning, PMIC calibration, and thermal resistance was not characterized.
- **Short generation length.** Each prompt generates at most 128 tokens. Longer generations would produce a more fully populated KV cache, potentially revealing stronger context-window effects than observed here.
- **PPL as a self-referential quality metric.** Perplexity measures the model’s internal confidence over its own outputs, not external correctness or factual accuracy. A comparison against an independent reference model or human evaluation would provide a more absolute quality signal.
- **Throttle flag interpretation.** `vcgencmd get_throttled` accumulates a historical bitmask within a boot session rather than reporting instantaneous state, slightly overstating the prevalence of throttling for later configurations in a session.

References

- [1] Nezami, Z., et al. “Generative AI on the Edge: Architecture and Performance Evaluation.” arXiv:2411.17712, 2024.
- [2] Yoon, K., et al. “LiteRT-Optimized INT8 LLM for Raspberry Pi4 Deployment.” ICCV Workshops, 2025.
- [3] Dettmers, T., et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.” NeurIPS, 2022.
- [4] Andrei, A. “llama-cpp-python: Python Bindings for llama.cpp.” <https://github.com/abetlen/llama-cpp-python>, 2023.
- [5] Sanei, D. “edge_ai_performance_profiling.” https://github.com/danielsanei/edge_ai_performance_profiling, 2026.